Aztec Logic Module Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

Farouk

qpzm

April 6, 2026

Contents

1 About Cyfrin **2**

2 Disclaimer **2**

3 Risk Classification **2**

4 Protocol Summary **2**

5 Audit Scope **7**

6 Executive Summary **7**

7 Findings **9**

7.1 Low Risk . . . 9

7.1.1 Malformed ACIR logic constraints can throw or hard-abort instead of failing the circuit normally 9

7.1.2 Non-constant logic operations discard `OriginTag` provenance and bypass debug tag checks 10

7.1.3 Missing Builder Context Validation in `create_logic_constraint` . . . 11

7.1.4 Constant-only logic constraints spuriously fail in write-VK mode . . . 12

7.2 Informational . . . 14

7.2.1 Witness indices in the ACIR logic bridge are only bounds-checked in debug builds . . . 14

7.2.2 Oversized Plookup Table for Sub-32-bit Chunks . . . 14

7.2.3 Operator Precedence Bug in DSL Logic Constraint Test Input . . . 17

7.2.4 Redundant Range Constraints Between ACIR and Logic Gate . . . 18

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

The `logic` module implements circuit-friendly bitwise AND and XOR operations for the Aztec rollup. Noir's `&` and `^` operators on unsigned integers compile to `BlackBoxFunc::AND` / `XOR` ACIR opcodes, which Barretenberg processes via plookup-based constraints.

Production usage includes `is_power_of_2` checks (`n & (n-1) == 0`) in rollup structure validation and `compute_subtree_sizes` (`num_leaves & subtree_size`) in unbalanced Merkle trees.

Constraint Architecture

The module uses precomputed lookup tables (LogUp argument, called "plookup" in the codebase for historical reasons) with a two-level decomposition:

1. Level 1 — 32-bit chunks (`logic.cpp`): The input is split into chunks of up to 32 bits. Each chunk is processed independently, and the results are reconstructed using positional scaling factors (chunk $\times 2^{32i}$).
2. Level 2 — 6-bit slices (plookup multi-table): Each 32-bit chunk is further decomposed into `[6, 6, 6, 6, 6, 2]`-bit slices. Each slice is looked up in a BasicTable of $64 \times 64 = 4096$ entries (for 6-bit slices) or $4 \times 4 = 16$ entries (for 2-bit slices). The plookup accumulator reconstruction proves that the slices reassemble to the original chunk.

The plookup system has two table layers:

- MultiTable (metadata): Describes how to decompose an input into slices. Created once at static initialization for all table types (`UINT8/16/32/64_XOR/AND`). Contains slice sizes, basic table IDs, and function pointers. Registered at static initialization (cheap — just metadata, not row data).
- BasicTable (actual rows): The lookup entries (e.g., $64 \times 64 = 4096$ rows for a 6-bit table). Created lazily by the circuit builder only when a plookup read references that table (`ultra_circuit_builder.cpp:512`). If no lookup uses a table, its rows are never generated.

3 distinct BasicTables per operation (XOR/AND):

BasicTable	Slice bits	Rows	Used by MultiTables
`UINT_XOR_SLICE_6_ROTATE_0`	6	64×64 = 4096	All (UINT8/16/32/64_XOR)
`UINT_XOR_SLICE_4_ROTATE_0`	4	16×16 = 256	UINT16_XOR, UINT64_XOR (last slice)
`UINT_XOR_SLICE_2_ROTATE_0`	2	4×4 = 16	UINT8_XOR, UINT32_XOR (last slice)

MultiTable slice layouts:

```
UINT64_XOR: [6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 4]  -> 11 sub-lookups using SLICE_6 ×10 + SLICE_4 ×1
UINT32_XOR: [6, 6, 6, 6, 6, 2]                  ->  6 sub-lookups using SLICE_6 ×5  + SLICE_2 ×1
UINT16_XOR: [6, 6, 4]                            ->  3 sub-lookups using SLICE_6 ×2  + SLICE_4 ×1
UINT8_XOR:  [6, 2]                               ->  2 sub-lookups using SLICE_6 ×1  + SLICE_2 ×1
```

`UINT8_XOR` and `UINT32_XOR` share the same BasicTables (`SLICE_6` and `SLICE_2`). Using both MultiTables in the same circuit does not add new BasicTable rows — only the number of sub-lookups differs.

Circuit Constraints

Runtime assertions (prover-side only, not circuit constraints):

ID	Code	Location	Purpose
A-1	`BB_ASSERT_LTE(num_bits, 252)`	`logic.cpp:40`	Prevents field overflow
A-2	`BB_ASSERT_GT(num_bits, 0U)`	`logic.cpp:41`	Prevents zero-iteration loop
A-3	`BB_ASSERT_LTE(a_native.get_msb(), num_bits-1)`	`logic.cpp:46`	Both-constant path: input fits in `num_bits`
A-4	`BB_ASSERT_LTE(b_native.get_msb(), num_bits-1)`	`logic.cpp:48`	Both-constant path: input fits in `num_bits`

Circuit constraints (enforced by verifier):

ID	Type	Location	Purpose
C-1	Plookup	`logic.cpp:93`	Per chunk: table membership + 32-bit range
C-2	Range	`logic.cpp:101`	Last chunk: tightens a to $[0, 2^{\text{chunk_size}})$
C-3	Range	`logic.cpp:102`	Last chunk: tightens b to $[0, 2^{\text{chunk_size}})$
C-4	Equality	`logic.cpp:111`	$a = \sum_i a_{\text{chunk}_i} \times 2^{32i}$
C-5	Equality	`logic.cpp:112`	$b = \sum_i b_{\text{chunk}_i} \times 2^{32i}$
D-1	Equality	`logic_constraint.cpp:27`	Binds result to ACIR output witness

C-1 expands into 6 sub-lookups per chunk via the UINT32_XOR/AND MultiTable. C-2/C-3 only fire when `chunk_size != 32`. The step size ($= 2^6 = 64$, from the MultiTable coefficient series, distinct from the LogUp challenge β) is the q_2 selector value used to extract raw slices from accumulator wires: $d = w[\text{row}] - \text{step} \times w[\text{row} + 1]$.

ID	Table	Slice	Entries	Step size (q_2)
P-1	`UINT_XOR/AND_SLICE_6_ROTATE_0`	bits [0:5]	4096	64
P-2	`UINT_XOR/AND_SLICE_6_ROTATE_0`	bits [6:11]	4096	64
P-3	`UINT_XOR/AND_SLICE_6_ROTATE_0`	bits [12:17]	4096	64

ID	Table	Slice	Entries	Step size (q_2)
P-4	`UINT_XOR/AND_SLICE_6_ROTATE_0`	bits [18:23]	4096	64
P-5	`UINT_XOR/AND_SLICE_6_ROTATE_0`	bits [24:29]	4096	64
P-6	`UINT_XOR/AND_SLICE_2_ROTATE_0`	bits [30:31]	16	0 (last row)

Total for 252-bit operation: 48 plookup sub-lookups (8×6) + 2 range constraints + 2 reconstruction equalities + 1 output binding = 53 logical constraints.

Note: "53 constraints" is the logical constraint count (distinct constraint operations), not the UltraHonk gate count. Each logical constraint maps to a different number of gates. Measured gate counts for the two production `num_bits` values:

`num_bits`	Chunks	Last chunk	Gates	Note
16	1	16	1375	High due to `create_range_constraint(16)`
32	1	32	6	Optimal — no range constraint needed

All cases use 2 BasicTables (SLICE_6: 4096 rows + SLICE_2: 16 rows = 4112 total). The `num_bits=32` case (6 gates) matches exactly 6 plookup sub-lookups with no range constraints. The `num_bits=16` case has high gate count because `create_range_constraint(16)` decomposes into many arithmetic gates.

A potential source of vulnerabilities is a prover providing chunks that individually satisfy the lookup but reconstruct to a different value than the original input. The module prevents this with:

- Reconstruction constraints (C-4, C-5): $a = \sum_i a_{\text{chunk}_i} \times 2^{32i}$ — proves the chunks reassemble to the original input.
- Range constraints (C-2, C-3): When the last chunk is smaller than 32 bits, explicit `create_range_constraint` calls prevent the prover from using values that exceed the intended bit width. For full 32-bit chunks, the plookup table membership itself provides implicit 32-bit range checking.

LogUp Lookup Argument

LogUp (Log-Derivative Lookup) is the lookup argument used in barretenberg's UltraHonk proving system. It replaces the original plookup grand-product approach with a sum-of-inverses identity. The codebase retains "plookup" in file and API names (`plookup_tables/`, `plookup_read`, `MultiTableId`) for historical reasons.

In the logic module, LogUp verifies that every 6-bit slice produced by the accumulator decomposition exists as a valid row in the XOR/AND BasicTable. Without LogUp, the prover could claim any result for `a & b` or `a ^ b`.

The LogUp argument proves:

$$\sum_i \frac{q_i}{L_i} = \sum_j \frac{c_j}{T_j}$$

- Left side: sum over all lookup gate rows (execution trace). $q_i = 1$ if row i is a lookup gate.
- Right side: sum over all table rows, weighted by read count c_j (how many times row j was looked up).
- L_i = lookup term (derived from wires + step sizes)
- T_j = table term (derived from committed table polynomials)

If every lookup entry exists in the table, the sums balance. If any lookup is fake (i.e., not in the table), the sum cannot balance with overwhelming probability over the random challenges.

Multi-column entries are compressed into single field elements using random challenges β and γ:

$$T = \gamma + \text{table}_1 + \beta \cdot \text{table}_2 + \beta^2 \cdot \text{table}_3 + \beta^3 \cdot \text{table_index}$$

$$L = \gamma + d_1 + \beta \cdot d_2 + \beta^2 \cdot d_3 + \beta^3 \cdot \text{table_index}$$

where $d_i = w_i[\text{row}] + (-\text{step_size}) \times w_i[\text{row} + 1]$ extracts raw slices from the accumulator wires. The accumulator stores a running sum, not the raw slice — for example, $w_1[\text{row } 0] = 222$ is the full accumulated value for column a, not the 6-bit slice 30. To recover the raw slice, the contribution of the remaining (higher) slices is subtracted out:

- $w_i[\text{row}]$ = current accumulator value
- $w_i[\text{row} + 1]$ = next row's accumulator ($w_{i,\text{shift}}$ — the same wire polynomial evaluated at the next row)
- $-$step_size = the q_2/q_m/q_c selector value ($= -64$ for all rows, 0 for the last row). The step size comes from the MultiTable coefficient series (base 64), not from the individual table's native base.

The subscript i in d_i refers to the wire column (1 = a, 2 = b, 3 = result), not the row. Each row extracts three slices — one per wire column.

Example (8-bit XOR, $222 \oplus 173 = 115$, Row 0):

- $d_1 = 222 + (-64) \times 3 = 30$ (a slice)
- $d_2 = 173 + (-64) \times 2 = 45$ (b slice)
- $d_3 = 115 + (-64) \times 1 = 51$ (result slice)
- LogUp checks $(30, 45, 51)$ against SLICE_6_XOR table: $30 \oplus 45 = 51$

The prover introduces a witness polynomial I:

$$I_i = \begin{cases} \frac{1}{L_i \cdot T_i} & \text{for active rows} \\ 0 & \text{for inactive rows} \end{cases}$$

I is a proper polynomial of degree $\leq N - 1$ that interpolates these pointwise values. It is not the rational function $1/(L(x) \cdot T(x))$.

LogUp Constraint Enumeration

ID	Constraint	Type	Degree	Per-row?	Purpose
S-0	$I_i \cdot L_i \cdot T_i - \text{inverse_exists}_i = 0$	Inverse correctness	4	Yes	I is correctly computed
S-1	$\sum_i (q_i \cdot I_i \cdot T_i - c_i \cdot I_i \cdot L_i) = 0$	Lookup identity	4	No (global)	Every lookup exists in the table
S-2	$\text{read_tag}_i^2 - \text{read_tag}_i = 0$	Boolean check	2	Yes	Prevents read_tag manipulation

Where inverse_exists $= q +$ read_tag $- q \cdot$ read_tag indicates whether a row is active (lookup gate or read table row).

- S-0: Active row forces $I = 1/(L \cdot T)$. Inactive row forces $I = 0$.
- S-1: The core identity — substituting $I = 1/(L \cdot T)$ yields $\sum[q/L - c/T] = 0$.
- S-2: Forces read_tag $\in \{0, 1\}$. Without this constraint, a prover could set read_tag $= 1 + \delta$ to fake lookups.

S-0 and S-2 are multiplied by a per-row scaling factor for enforcement at every row (linearly independent). S-1 is a global sum (linearly dependent — no per-row scaling). All three subrelations are separated by α powers via `scale_univariates`.

How Logic Gates Become LogUp Constraints

```
logic.cpp: create_logic_constraint(a, b, 32, is_xor)
  → plookup.cpp: read_from_2_to_1_table(UINT32_AND, a_chunk, b_chunk)
    → plookup_tables.cpp: get_lookup_accumulators()
      Native: decompose into [6,6,6,6,6,2] slices, compute AND/XOR per slice
      Build accumulator values bottom-up
    → ultra_circuit_builder.cpp: create_gates_from_plookup_accumulators()
      For each sub-lookup (6 total):
        - get_table() → lazy BasicTable creation
        - table.lookup_gates.emplace_back() → record for read counts
        - populate_wires(a_acc, b_acc, result_acc, zero)
        - q_lookup = 1, q_3 = table_index, q_2/q_m/q_c = -step_size
  → Prover: compute_logderivative_inverse()
      $I[i] = 1/(L_i \cdot T_i)$ for all active rows
  → Verifier: accumulate()
      Check S-0, S-1, S-2 at each row via Sumcheck
```

For one `create_logic_constraint(a, b, 32, is_xor)` call:

Component	Count
Lookup gate rows (q_lookup=1)	6
Table rows used	6
S-0 checks (inverse correctness)	12 (6 lookup + 6 table rows)
S-1 terms (lookup identity)	12 (6 × q/L + 6 × c/T)
S-2 checks (boolean)	6 (table rows with read_tag)

Plus the logic module's own constraints: C-2/C-3 (range), C-4/C-5 (reconstruction), D-1 (output binding).

Soundness Argument

The prover cannot fake a bitwise operation because:

1. LogUp (S-1) ensures that every `(a_slice, b_slice, result_slice)` triple exists in the BasicTable.
2. BasicTable membership implicitly range-checks each slice (SLICE_6: $[0, 63]$, SLICE_2: $[0, 3]$).
3. Accumulator step sizes (in q_2/q_m/q_c) ensure that slices are extracted correctly from the wires.
4. Reconstruction constraints (C-4/C-5) ensure that the slices reassemble to the original input.
5. The boolean check (S-2) prevents read_tag manipulation that could create fake table credits.

Key Polynomials

Polynomial	Source	Role
`q_lookup`	Gate selector	1 on lookup gate rows
`w_l, w_r, w_o`	Wires (accumulator values)	Store accumulated sums
`w_l_shift, w_r_shift, w_o_shift`	Next-row wire values	Used with step sizes to extract slices
`q_2`, `q_m`, `q_c`	Selectors	`-step_size` for columns 1, 2, 3
`q_3`	Selector	`table_index`
`table_1, table_2, table_3, table_4`	Committed from BasicTable	Table row data + identifier
`lookup_inverses`	Prover witness	`I = 1/(L × T)` at active rows

Polynomial	Source	Role
`lookup_read_counts`	Prover witness	How many times each table row was read
`lookup_read_tags`	Prover witness	Boolean: 1 if table row was read

5 Audit Scope

The audit scope was limited to:

```
aztec-packages/barretenberg/cpp/src/barretenberg
 stdlib_circuit_builders/plookup_tables/uint.hpp
 stdlib/primitives/logic/logic.cpp
 dsl/acir_format/logic_constraint.cpp
 stdlib/primitives/logic/logic.hpp
 dsl/acir_format/logic_constraint.hpp
```

6 Executive Summary

Over the course of 4 days, the Cyfrin team conducted an audit on the Aztec Logic Module code provided by Aztec Labs. The findings consist of 4 Low severity issues with the remainder being informational.

The Aztec Logic module implements bitwise AND and XOR constraint generation for Barretenberg's circuit standard library, decomposing operations into 32-bit plookup table reads with chunk reconstruction verification. The security model depends on correct range constraints bounding each chunk, faithful plookup table implementations, and reconstruction assertions that bind decomposed chunks back to original operands.

Our review focused on the stdlib constraint generation logic, the plookup multi-table construction for 32-bit AND/XOR operations, the DSL bridge layer converting ACIR opcodes to circuit constraints, and the interaction between these components across both the UltraCircuitBuilder and MegaCircuitBuilder backends.

The audit revealed a well-implemented constraint generation module with sound core logic. The chunked plookup approach with explicit final-chunk range constraints correctly addresses the known historical soundness concern where missing range constraints in 6-bit table decomposition allowed values to be decomposed into multiple potential elements. The low severity findings relate to edge cases in the DSL integration layer: constant-only logic constraints fail spuriously during verification key generation due to dummy witness values, the `OriginTag` provenance system is bypassed for non-constant operations because chunk witnesses default to free-witness tags, cross-builder operand misuse lacks early validation, and malformed ACIR inputs trigger host-level aborts rather than graceful circuit failures. The informational findings identify efficiency opportunities in table selection for sub-32-bit operations, redundant range constraints between the ACIR pipeline and the logic gate, an operator precedence bug in the DSL test suite that weakens boundary coverage, and debug-only bounds checking on witness indices in the ACIR bridge.

Summary

Project Name	Aztec Logic Module
Repository	aztec-packages
Commit	e4712cda8def...
Audit Timeline	Mar 16th - Mar 19th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	4
Informational	4
Gas Optimizations	0
Total Issues	8

Summary of Findings

[L-1] Malformed ACIR logic constraints can throw or hard-abort instead of failing the circuit normally	Acknowledged
[L-2] Non-constant logic operations discard `OriginTag` provenance and bypass debug tag checks	Resolved
[L-3] Missing Builder Context Validation in `create_logic_constraint`	Resolved
[L-4] Constant-only logic constraints spuriously fail in write-VK mode	Acknowledged
[I-1] Witness indices in the ACIR logic bridge are only bounds-checked in debug builds	Acknowledged
[I-2] Oversized Plookup Table for Sub-32-bit Chunks	Acknowledged
[I-3] Operator Precedence Bug in DSL Logic Constraint Test Input	Resolved
[I-4] Redundant Range Constraints Between ACIR and Logic Gate	Acknowledged

7 Findings

7.1 Low Risk

7.1.1 Malformed ACIR logic constraints can throw or hard-abort instead of failing the circuit normally

Description: `create_logic_gate(...)` in `dsl/acir_format/logic_constraint.cpp` forwards `num_bits` and constant operands directly into `logic<Builder>::create_logic_constraint(...)`.

```
template <typename Builder>
void create_logic_gate(Builder& builder,
                       const WitnessOrConstant<bb::fr> a,
                       const WitnessOrConstant<bb::fr> b,
                       const uint32_t result,
                       const size_t num_bits,
                       const bool is_xor_gate)
{
    using field_ct = bb::stdlib::field_t<Builder>;

    field_ct left = to_field_ct(a, builder);
    field_ct right = to_field_ct(b, builder);

    field_ct computed_result = bb::stdlib::logic<Builder>::create_logic_constraint(left, right,
    ↪  num_bits, is_xor_gate);
    field_ct acir_result = field_ct::from_witness_index(&builder, result);
    computed_result.assert_equal(acir_result);
}
```

The logic gadget enforces `0 < num_bits <= 252` and constant-operand bit bounds with `BB_ASSERT_*` in `stdlib/primitives/logic/logic.cpp`, rather than through a builder-failure path. Those assertions route through `common/assert.hpp`, `common/assert.cpp`, and `env/throw_or_abort_impl.cpp`, which means malformed logic constraints throw in exception-enabled builds and abort the process in `BB_NO_EXCEPTIONS` builds.

```
// ensure the number of bits doesn't exceed field size and is not negative
BB_ASSERT_LTE(num_bits, grumpkin::MAX_NO_WRAP_INTEGER_BIT_LENGTH);
BB_ASSERT_GT(num_bits, 0U);

if (a.is_constant() && b.is_constant()) {
    uint256_t a_native = static_cast<uint256_t>(a.get_value());
    uint256_t b_native = static_cast<uint256_t>(b.get_value());
    BB_ASSERT_LTE(
        a_native.get_msb(), num_bits - 1, "field_t: Left operand in logic gate exceeds specified bit
        ↪  length");
    BB_ASSERT_LTE(
        b_native.get_msb(), num_bits - 1, "field_t: Right operand in logic gate exceeds specified bit
        ↪  length");
```

```
// Native implementation of throw_or_abort
extern "C" void throw_or_abort_impl [[noreturn]] (const char* err)
{

#ifdef STACKTRACES
    // Use backward library to print stack trace
    backward::StackTrace trace;
    trace.load_here(32);
    backward::Printer{}.print(trace);
#endif
#ifndef BB_NO_EXCEPTIONS
    throw std::runtime_error(err);
#else
    abort_with_message(err);
```

```
#endif
}
```

Impact: Malformed or untrusted ACIR logic opcodes can terminate circuit construction at the host level instead of producing a normal unsatisfied-constraint result or explicit rejected-opcode error. This is an availability and robustness issue. The worst case is `BB_NO_EXCEPTIONS`, where the same path becomes a hard process abort.

Recommended Mitigation: Validate `LogicConstraint` inputs in the ACIR layer before calling the stdlib gadget. Downgrade malformed `num_bits` and constant-operand violations to builder failure or explicit rejected-opcode errors, rather than relying on host-level assertions in the logic gadget.

Aztec: Acknowledged, It is true that `create_logic_constraint` uses `BB_ASSERT` (which aborts) rather than throwing an exception when `num_bits` is invalid. `BB_ASSERT` is the standard pattern throughout barretenberg for structural invariant checks. Additionally, the `num_bits` value originates from Noir's `IntegerBitSize` enum, which restricts values to {1, 8, 16, 32, 64, 128} at the compiler level, so invalid values cannot reach this code through normal operation.

7.1.2 Non-constant logic operations discard `OriginTag` provenance and bypass debug tag checks

Description: The non-constant path in `stdlib/primitives/logic/logic.cpp` rebuilds both operands from fresh chunk witnesses:

```
field_pt a_chunk = witness_pt(ctx, left_chunk);
field_pt b_chunk = witness_pt(ctx, right_chunk);
```

Those derived witnesses default to free-witness provenance via the field/witness construction path reflected in `stdlib/primitives/field/field.cpp`.

```
template <typename Builder>
field_t<Builder> field_t<Builder>::from_witness_index(Builder* ctx, const uint32_t witness_index)
{
    field_t<Builder> result(ctx);
    result.witness_index = witness_index;
    // Since this is now a witness (not a constant), set the free witness tag
    // The caller should set the appropriate tag if this element has a known provenance
    result.set_free_witness_tag();
    return result;
}
```

The logic result is then derived entirely from those fresh witnesses, and the final `assert_equal` calls only bind values; they do not restore the original provenance and temporarily suppress origin-tag checking for witness-to-witness equality in `stdlib/primitives/field/field.cpp:940-980`. As a result, the `OriginTag` guard in `transcript/origin_tag.cpp` is never evaluated for the original non-constant inputs.

```
// A free witness element should not interact with an element that has an origin
if (tag_a.is_free_witness()) {
    if (!tag_b.is_free_witness() && !tag_b.is_empty()) {
        throw_or_abort("A free witness element should not interact with an element that has an origin");
    } else {
        // If both are free witnesses or one of them is empty, just use tag_a
        *this = tag_a;
        return;
    }
}
if (tag_b.is_free_witness()) {
    if (!tag_a.is_free_witness() && !tag_a.is_empty()) {
        throw_or_abort("A free witness element should not interact with an element that has an origin");
    } else {
        // If both are free witnesses or one of them is empty, just use tag_b
        *this = tag_b;
        return;
    }
```

```
}
```

Impact: In debug builds, the logic gadget can combine transcript-derived values with free witnesses, or values from different transcript sources, without triggering the `OriginTag` security checks that ordinary field arithmetic would enforce. This does not change proof soundness in release builds, where origin-tag checks are compiled out, but it weakens the active debug-time defense intended to catch unsafe Fiat-Shamir interactions.

Recommended Mitigation: Preserve provenance in the witness path exactly as other stdlib gadgets do. At minimum, compute `OriginTag(a.get_origin_tag(), b.get_origin_tag())` once in `create_logic_constraint` before chunking and assign the merged tag to the returned value. Preferably also tag derived chunk/result witnesses consistently so intermediate gadget outputs do not revert to free-witness provenance. When converting constants to fixed witnesses in the mixed path, explicitly preserve the original constant tag instead of accepting the default free-witness tag.

Aztec: Fixed in 0f3ca1c.

Cyfrin: Verified.

7.1.3 Missing Builder Context Validation in `create_logic_constraint`

Description: In `stdlib/primitives/logic/logic.cpp:80`, when both `a` and `b` are witnesses, the function extracts the builder context from only one operand without validating that both operands belong to the same builder:

```
// Both a and b are witnesses at this point, so they have the same context.
Builder* ctx = a.get_context();
```

There is no check that `a.get_context() == b.get_context()`. If a developer passes operands from different builders, the function would:

1. Create `b_chunk` witnesses in `a`'s builder (`witness_pt(ctx, right_chunk)`)
2. Accumulate `b_accumulator` in `a`'s builder
3. Call `b.assert_equal(b_accumulator)` — comparing a witness in `b`'s builder with one in `a`'s builder — undefined behavior

Related Occurrence (Same Root Cause):

The same "pick one context and assume the other matches" pattern exists in the plookup read layer.

In `stdlib/primitives/plookup/plookup.cpp:19`, `plookup_read<Builder>::get_lookup_accumulators(...)` selects a `ctx` from either `key_a` or `key_b`:

```
Builder* ctx = key_a.get_context() ? key_a.get_context() : key_b.get_context();
```

and then uses witness indices from both operands (for the variable path) without validating that both keys belong to the same builder. If `key_a` and `key_b` come from different builders, this can similarly lead to undefined behavior (e.g., passing a witness index allocated in one builder into `create_gates_from_plookup_accumulators` on another builder).

Impact: Low. This is a developer error (mixing builders), not a prover attack. The prover does not control which builder objects are used — this is determined at circuit construction time. If triggered, it would cause undefined behavior or assertion failures during circuit construction, not during verification.

Proof of Concept:

```
// Missing builder context validation.
// When operands come from different builders, create_logic_constraint uses only a's context.
// b's chunks get created in the wrong builder, and b.assert_equal() compares across builders.
// This test documents the undefined behavior - it should ideally be caught by an assertion.
TYPED_TEST(LogicTest, CrossBuilderContextMismatch)
{
    STDLIB_TYPE_ALIASES
```

```
    auto builder_1 = Builder();
    auto builder_2 = Builder();

    field_ct a = witness_ct(&builder_1, uint256_t(42));
    field_ct b = witness_ct(&builder_2, uint256_t(17));

    // a.get_context() != b.get_context(), but no validation exists.
    // The function will use builder_1's context for everything,
    // then call b.assert_equal() which compares a witness in builder_2
    // with an accumulator in builder_1.
    EXPECT_NE(a.get_context(), b.get_context());

    // This call exhibits undefined behavior due to cross-builder operands.
    // Depending on implementation, it may:
    // - Silently produce a broken circuit
    // - Crash with an out-of-range witness index
    // - Trigger an assertion in debug builds
    // Any of these outcomes confirms the missing validation.
    EXPECT_ANY_THROW(stdlib::logic<Builder>::create_logic_constraint(a, b, 32, true));
}
```

Test result: The error is caught downstream by `assert_equal()` with: `"Pointers refer to different builder objects!"`. An early validation in `create_logic_constraint` would provide a clearer error at the point of misuse.

Recommended Mitigation: Add context validation before the main loop. Since constant cases are already handled above (lines 62-77), at this point both `a` and `b` are witnesses with non-null contexts:

Option 1 — explicit assertion:

```
ASSERT(a.get_context() == b.get_context() && "logic constraint: operands belong to different builders");
Builder* ctx = a.get_context();
```

Option 2 — use existing `validate_context` utility from `field.hpp`:

```
Builder* ctx = validate_context(a.get_context(), b.get_context());
```

Consider applying the same pattern in `plookup_read<Builder>::get_lookup_accumulators(...)` before using witness indices from both keys.

Aztec: Fixed in 249bfdc.

Cyfrin: Verified.

7.1.4 Constant-only logic constraints spuriously fail in write-VK mode

Description: When generating a Verification Key without real witnesses, `create_circuit(...)` zero-fills all witness slots (`dsl/acir_format/acir_format.cpp:203`).

```
const bool is_write_vk_mode = witness.empty();

if (!is_write_vk_mode) {
    BB_ASSERT_EQ(witness.size(),
                 constraints.max_witness_index + 1,
                 "ACIR witness size (" << witness.size() << ") does not match max witness index + 1 ("
                                       << (constraints.max_witness_index + 1) << ").");
} else {
    witness.resize(constraints.max_witness_index + 1, 0);
}
```

For a constant-only logic constraint (e.g., `5 AND 3 = 1`), the stdlib fast path computes the correct result as a constant `field_ct(1)`. But `create_logic_gate` (`dsl/acir_format/logic_constraint.cpp:26-27`) then asserts this constant equals the result witness — which holds the dummy value `0`:

```
field_ct acir_result = field_ct::from_witness_index(&builder, result);  // witness value = 0 (dummy)
computed_result.assert_equal(acir_result);                               // constant 1 != witness 0
```

Since `computed_result` is constant and `acir_result` is a witness, `field_t::assert_equal` (`field.cpp:948-951`) calls `assert_equal_constant()` (`ultra_circuit_builder.hpp:416-420`), which compares the witness value against the constant and calls `builder.failure(msg)` when they don't match.

The ACIR arithmetic constraint handler avoids this by checking `!builder.is_write_vk_mode()` before calling `failure()` (`arithmetic_constraints.cpp:43`). The logic constraint path has no such guard.

Impact: VK generation is broken for any circuit containing a constant-only AND/XOR whose result is non-zero. The builder is marked as failed even though the circuit structure is valid and the VK would be correct. Downstream tooling that checks `builder.failed()` will reject the VK or abort the pipeline. This affects availability of VK generation, not proof soundness.

Recommended Mitigation: Guard the value-based check in `create_logic_gate` against write-VK mode, matching the arithmetic constraint pattern.

Aztec: Fixed in a4837fe.

Cyfrin: Verified.

7.2 Informational

7.2.1 Witness indices in the ACIR logic bridge are only bounds-checked in debug builds

Description: `create_logic_gate` passes three witness indices to the builder without any bounds validation:

```
// dsl/acir_format/logic_constraint.cpp:22-27
field_ct left = to_field_ct(a, builder);
field_ct right = to_field_ct(b, builder);

field_ct computed_result = bb::stdlib::logic<Builder>::create_logic_constraint(left, right, num_bits,
↪  is_xor_gate);
field_ct acir_result = field_ct::from_witness_index(&builder, result);
computed_result.assert_equal(acir_result);
```

For non-constants, `to_field_ct` stores the index without checking it:

```
// dsl/acir_format/witness_constant.hpp:44-45
return field_ct::from_witness_index(&builder, input.index);
```

When the index is later dereferenced (e.g., `get_value()` during chunking, or `assert_equal` during normalization), it reaches `get_variable()`:

```
// stdlib_circuit_builders/circuit_builder_base.hpp:159-163
inline FF get_variable(const uint32_t index) const
{
    BB_ASSERT_DEBUG(real_variable_index.size() > index);      // compiled out in release
    BB_ASSERT_DEBUG(variables.size() > real_variable_index[index]); // compiled out in release
    return variables[real_variable_index[index]];             // raw OOB if index is bad
}
```

`BB_ASSERT_DEBUG` compiles to nothing under `NDEBUG` (release builds). The normal ACIR parser computes `max_witness_index` from opcodes, so well-formed parsed ACIR stays within bounds. But `create_circuit` sizes the witness array to `max_witness_index + 1` without re-checking every individual index embedded in constraints. A manually constructed or post-parse mutated `AcirFormat` with an index exceeding that bound will silently read out of bounds in release builds.

Not a proof-soundness issue for well-formed parsed ACIR. Malformed in-memory `AcirFormat` structs (direct C++ callers, post-parse mutations, or a parser bug in `max_witness_index` tracking) can trigger out-of-bounds reads in release builds, leading to crashes or garbage witness values.

Recommended Mitigation: Validate witness indices against the builder's variable bounds in `to_field_ct` / `from_witness_index` regardless of `NDEBUG`, or promote the debug assertions in `get_variable()` to unconditional checks.

Aztec: Acknowledged, as it will be fixed later. The witness index bounds checks in `get_variable()` use `BB_ASSERT_DEBUG`, which is compiled out in release builds. This is known and it affects every circuit component that calls `get_variable()`, not just the logic gadget. We are discussing this the noir team if it can be exploited in any meaningful way. Regardless, we will enable these asserts in release mode soon after ensuring no significant hit in performance because of the asserts.

7.2.2 Oversized Plookup Table for Sub-32-bit Chunks

Description: In `stdlib/primitives/logic/logic.cpp:111`, the multi-table ID is always `UINT32_XOR` or `UINT32_AND` regardless of the actual `chunk_size`:

```
const auto multi_table_id = is_xor_gate ? plookup::MultiTableId::UINT32_XOR :
↪  plookup::MultiTableId::UINT32_AND;
```

The `UINT32` multi-table decomposes each input into 6 sub-lookups (`[6, 6, 6, 6, 6, 2]` bits). When the last chunk has `chunk_size  8`, smaller tables already exist (`types.hpp:105-112`):

```
UINT8_XOR,   // 2 sub-lookups: [6, 2]
UINT16_XOR,  // 3 sub-lookups: [6, 6, 4]
UINT32_XOR,  // 6 sub-lookups: [6, 6, 6, 6, 6, 2]
```

For example, when `num_bits = 1` (so `chunk_size = 1`), the code performs 6 sub-lookups where 5 look up `(0, 0, 0)` — valid but wasteful. Using `UINT8_XOR` would need only 2 sub-lookups for the same result.

Additionally, a `UINT8` table implicitly proves 8-bit range, so the explicit `create_range_constraint` at line 127 could be skipped for `chunk_size  8`, saving further gates.

Impact: Informational. For 16-bit operations, using `UINT16` instead of `UINT32` would reduce total gates from 1376 to 10 per call by eliminating the explicit `create_range_constraint(16)`, improving proving cost.

Current production usage comparison:

num_bits	Current table	Sub-lookups	Range constraint	Optimized table	Sub-lookups	Range constra
16	UINT32	6	`create_range_constraint(16)`	UINT16	3	Not needed (in
32	UINT32	6	Not needed (implicit)	UINT32	6	Not needed (in

The Noir rollup circuits currently use AND operations with `num_bits = 16` and `32`:

Noir function	Circuit	num_bits	Op
`is_power_of_2` (`n & (n-1) == 0`)	Tx Merge Rollup, Block Root Rollup	16 (u16)	AND
`is_power_of_2` (`n & (n-1) == 0`)	Tx Merge Rollup, Block Root Rollup	32 (u32)	AND
`compute_subtree_sizes` (`num_leaves & subtree_size`)	Tx Base Rollup	32 (u32)	AND

For `num_bits = 32`, every chunk is exactly 32 bits — no wasted sub-lookups. For `num_bits = 16`, the savings are significant: using `UINT16_AND` instead of `UINT32_AND` eliminates the explicit `create_range_constraint(16)` (which the UINT16 table proves implicitly), saving 1372 gates per call.

Measured gate counts for a single 16-bit AND lookup:

Approach	Gates	Total gates	Tables	Table rows
UINT32_AND + `create_range_constraint(16)`	1375	1376	2	4112 (SLICE_6 + SLICE_2)
UINT16_AND (no range constraint needed)	3	4	2	4352 (SLICE_6 + SLICE_4)

The dominant cost in the current approach is `create_range_constraint(16)`, which decomposes into ~1369 arithmetic gates internally. The UINT16 table's 3 sub-lookups implicitly prove a 16-bit range, making that explicit constraint unnecessary.

Since production circuits use both `num_bits = 32` and `num_bits = 16`, the circuit would need both `UINT32_AND` and `UINT16_AND` tables. The combined table cost:

Configuration	Total gates	Tables	Table rows
Current: UINT32_AND only	1376	2	4112 (SLICE_6: 4096 + SLICE_2: 16)
Proposed: UINT32_AND + UINT16_AND	10	3	4368 (SLICE_6: 4096 + SLICE_2: 16 + SLICE_4: 256)

The additional cost is 256 table rows (one `SLICE_4` BasicTable), a one-time overhead to save 1372 gates per 16-bit logic call.

Proof of Concept:

```
// Compare UINT32 vs UINT16 table for 16-bit lookups.
// UINT32 = 6 sub-lookups + explicit range constraint.
// UINT16 = 3 sub-lookups + implicit 16-bit range (no extra range constraint needed).
TYPED_TEST(LogicTest, CompareUint32VsUint16Table)
{
    STDLIB_TYPE_ALIASES
    using plookup_read = stdlib::plookup_read<Builder>;

    // Approach 1: Current code - UINT32_AND + range constraint
    {
        auto builder = Builder();
        field_ct a = witness_ct(&builder, uint256_t(0xABCD));
        field_ct b = witness_ct(&builder, uint256_t(0x1234));

        size_t gates_before = builder.num_gates();
        field_ct result = plookup_read::read_from_2_to_1_table(plookup::MultiTableId::UINT32_AND, a, b);
        a.create_range_constraint(16, "range");
        b.create_range_constraint(16, "range");
        size_t gates_after = builder.num_gates();

        EXPECT_EQ(uint256_t(result.get_value()), uint256_t(0xABCD & 0x1234));
        std::cout << "UINT32_AND + range_constraint(16): gates=" << (gates_after - gates_before)
                  << "  total_gates=" << builder.num_gates()
                  << "  tables=" << builder.get_num_lookup_tables()
                  << "  table_rows=" << builder.get_tables_size() << "\n";

        EXPECT_TRUE(CircuitChecker::check(builder));
    }

    // Approach 2: Both UINT32_AND + UINT16_AND in the same circuit (production scenario).
    {
        auto builder = Builder();
        field_ct a32 = witness_ct(&builder, uint256_t(0xDEADBEEF));
        field_ct b32 = witness_ct(&builder, uint256_t(0x12345678));
        field_ct a16 = witness_ct(&builder, uint256_t(0xABCD));
        field_ct b16 = witness_ct(&builder, uint256_t(0x1234));

        plookup_read::read_from_2_to_1_table(plookup::MultiTableId::UINT32_AND, a32, b32);
        plookup_read::read_from_2_to_1_table(plookup::MultiTableId::UINT16_AND, a16, b16);

        std::cout << "UINT32_AND + UINT16_AND combined:  "
                  << "total_gates=" << builder.num_gates()
                  << "  tables=" << builder.get_num_lookup_tables()
                  << "  table_rows=" << builder.get_tables_size() << "\n";

        EXPECT_TRUE(CircuitChecker::check(builder));
    }
}
```

Test output:

```
UINT32_AND + range_constraint(16): gates=1375  total_gates=1376  tables=2  table_rows=4112
UINT32_AND + UINT16_AND combined:  total_gates=10  tables=3  table_rows=4368
```

Recommended Mitigation: Consider selecting the smallest available multi-table that fits `chunk_size` (e.g., `UINT8` for `chunk_size  8`, `UINT16` for `chunk_size  16`).

Current production usage:

- `num_bits = 16` (math.nr:is_power_of_2, u16): Switching to `UINT16_AND` would save 1372 gates per call by eliminating the explicit `create_range_constraint(16)`.
- `num_bits = 32` (math.nr:is_power_of_2_u32, unbalanced_merkle_tree.nr:compute_subtree_sizes, u32): No change needed — `UINT32_AND` is already the optimal table.

Aztec: Acknowledged, The code always uses `UINT32_XOR`/`UINT32_AND` plookup tables even when the last chunk is smaller than 32 bits. Smaller tables (UINT8, UINT16) exist and would use fewer sub-lookups. However, this is a minor optimization that would change the circuit structure, which we want to avoid at this point. The existing explicit range constraint on the last chunk ensures correctness regardless of table size.

7.2.3 Operator Precedence Bug in DSL Logic Constraint Test Input

Description: In dsl/acir_format/logic_constraint.test.cpp:76-77, the test creates input values with a misleading comment:

```
bb::fr lhs = FF(static_cast<uint256_t>(1) << num_bits - 1); // All bits from 0 to num_bits-1 are set
bb::fr rhs = FF(static_cast<uint256_t>(1) << num_bits - 1); // All bits from 0 to num_bits-1 are set
```

Due to C++ operator precedence, `-` binds tighter than `<<`, so `1 << num_bits - 1` evaluates as `1 << (num_bits - 1)`. This produces a single-bit value (only bit `num_bits-1` set), not an all-bits-set value.

The comment says "All bits from 0 to num_bits-1 are set", which would require `(1 << num_bits) - 1`.

num_bits	Comment intent: `(1 << num_bits) - 1`	Actual: `1 << (num_bits - 1)`
1	`0b1` = 1	`0b1` = 1 (same by coincidence)
16	`0b1111111111111111` = 65535	`0b1000000000000000` = 32768
128	`0xFFFF...FFFF` (128 bits)	`0x8000...0000` (1 bit set)
252	`0xFFF...FFF` (252 bits)	`0x800...000` (1 bit set)

Since `lhs == rhs` in both cases, XOR always produces 0 and AND always produces `lhs`. This means the test only exercises:

- A single-bit input pattern (sparse), not an all-bits-set pattern (dense)
- XOR result always 0, AND result always equals input

The intended all-bits-set pattern would test the maximum value in range, which is a more interesting boundary case for the range constraints.

Impact: Informational. The test still exercises a valid input and passes. However, the test coverage is weaker than intended: the all-bits-set boundary case (maximum value within num_bits range) is not tested.

Proof of Concept: All 128 DSL logic constraint tests pass with the current (misleading) values:

```
$ ./bin/dsl_tests --gtest_filter="*LogicConstraint*"
[==========] 128 tests from 64 test suites ran. (1776 ms total)
[  PASSED  ] 128 tests.
```

Test matrix: 4 constancy modes (None, Input1, Input2, Both) × 4 num_bits (1, 16, 128, 252) × 2 operations (AND, XOR) × 2 builders (Ultra, Mega) × 2 test types (VK, Tampering) = 128 tests.

Recommended Mitigation: Fix the expression to match the comment — test the all-bits-set value:

```
bb::fr lhs = FF((static_cast<uint256_t>(1) << num_bits) - 1); // All bits from 0 to num_bits-1 are set
bb::fr rhs = FF((static_cast<uint256_t>(1) << num_bits) - 1); // All bits from 0 to num_bits-1 are set
```

Aztec: Fixed in 2a597fb.

Cyfrin: Verified.

7.2.4 Redundant Range Constraints Between ACIR and Logic Gate

Description: When a Noir circuit uses a bitwise operation like `a: u8 ^ b: u8`, the Noir compiler emits both range checks and the XOR opcode:

```
Opcode 0: BlackBoxFunc::RANGE(Witness[0], 8)   <- compiler-inserted range check on input a
Opcode 1: BlackBoxFunc::RANGE(Witness[1], 8)   <- compiler-inserted range check on input b
Opcode 2: BlackBoxFunc::XOR(Witness[0], Witness[1], num_bits=8, output=Witness[3])
```

Barretenberg processes these as separate constraints. The RANGE opcodes call `create_range_constraint(8)` on each input. Then inside `create_logic_constraint` (`logic.cpp:101-102`), when `chunk_size != 32`, the logic gate adds its own range constraints on the same inputs:

```
if (chunk_size != 32) {
    a_chunk.create_range_constraint(chunk_size, "stdlib logic: bad range on final chunk of left
    ↪  operand");
    b_chunk.create_range_constraint(chunk_size, "stdlib logic: bad range on final chunk of right
    ↪  operand");
}
```

For `num_bits = 8`, the same `create_range_constraint(8)` is called twice on each input — once by the ACIR RANGE opcode and once inside the logic gate. These are redundant: the first range check already proves the input is in `[0, 255]`.

Proof of Concept: Verified with a u8 XOR Noir circuit (`audit/my-report/poc/xor-circuit`):

```
fn main(a: u8, b: u8) -> pub u8 {
    a ^ b
}
```

Compiled ACIR opcodes (`python3 decode_acir.py target/xor_test.json`):

```
Function: main, Witnesses: 3, Opcodes: 4
  Opcode 0: {'BlackBoxFuncCall': {'RANGE': [{'Witness': 0}, 8]}}
  Opcode 1: {'BlackBoxFuncCall': {'RANGE': [{'Witness': 1}, 8]}}
  Opcode 2: {'BlackBoxFuncCall': {'XOR': [{'Witness': 0}, {'Witness': 1}, 8, 3]}}
  Opcode 3: {'AssertZero': [[], [[b'\x00...01', 2], [b'\x30...00', 3]], b'\x00...00']}
```

Barretenberg constraint trace showing the redundancy:

```
Opcode 0: RANGE(Witness[0], 8)
  → create_range_constraint(8)            ← first range check on a

Opcode 1: RANGE(Witness[1], 8)
  → create_range_constraint(8)            ← first range check on b

Opcode 2: XOR(Witness[0], Witness[1], 8, output=Witness[3])
  → create_logic_constraint(a, b, 8, true)
    → plookup UINT32_XOR (6 sub-lookups)
    → create_range_constraint(8) for a_chunk  ← redundant
    → create_range_constraint(8) for b_chunk  ← redundant
    → a.assert_equal(a_accumulator)
    → b.assert_equal(b_accumulator)
  → computed_result.assert_equal(acir_result)
```

End-to-end proof verified successfully with `a = 222 (0xDE)`, `b = 173 (0xAD)`, output `115 (0x73)`.

Impact: Informational. No soundness risk. The extra range constraints cost gates unnecessarily. For `num_bits = 8`, each redundant `create_range_constraint(8)` adds gates to the circuit.

Note: if the Oversized Plookup Table for Sub-32-bit Chunks optimization is implemented (using `UINT8_XOR` for 8-bit chunks), the logic gate's internal range constraint would be eliminated entirely (implicit in the table), making this issue moot for `chunk_size <= 8`.

Recommended Mitigation: No immediate action required. This is an informational observation about redundant constraints in the current pipeline.

Aztec: Acknowledged, there could be cases of redundant range constraints (on the noir side as well as barretenberg) but we prefer to have redundancy over missing a range constraint. This was a good find but we decided to not use the optimisation.